

```

---
title: Season Detail
toc: false
---

<link rel="stylesheet" href="./styles/App.css">

```js
import {OVERRIDE_YEAR, RIVER_MILE} from "./config.js";

```js
// ?? Unit conversion ?????????????????????????????????????????????????????????
const toF = c => c * 9 / 5 + 32;

```js
const raw = await FileAttachment("./data/RM15_water_temp.csv").text();
const parsed = d3.csvParse(raw, d => ({
 date: d3.timeParse("%Y-%m-%d")(d.date),
 tmp: d.tmp === "" ? null : toF(+d.tmp),
 cfs: d.cfs === "" ? null : +d.cfs
}));

```js
// ?? Season detection ?????????????????????????????????????????????????????????
// currentYear is auto-detected as the newest year present in the data
// (falling back to OVERRIDE_YEAR from config.js if it's set), and
// priorYear always follows one year behind it. This page's filename
// and title are stable ("season.md" / "Season Detail"), so there is
// nothing else to rename or edit here when the season changes.
const yearsInData = Array.from(new Set(
  parsed.filter(d => d.date !== null).map(d => d.date.getFullYear())
));
const currentYear = OVERRIDE_YEAR ?? d3.max(yearsInData);
const priorYear = currentYear - 1;

```js
const dataCurrent = parsed
 .filter(d => d.date?.getFullYear() === currentYear && d.tmp !== null)
 .sort((a, b) => a.date - b.date);

const threshold = toF(15.5);
const latest = dataCurrent[dataCurrent.length - 1];
const seasonHigh = d3.max(dataCurrent, d => d.tmp);
const seasonHighDate = dataCurrent.find(d => d.tmp === seasonHigh)?.date;
const daysAbove = dataCurrent.filter(d => d.tmp >= threshold).length;

```js
// 7-day trend
const last7 = dataCurrent.slice(-7);
const prior7 = dataCurrent.slice(-14, -7);
const last7Avg = d3.mean(last7, d => d.tmp);
const prior7Avg = d3.mean(prior7, d => d.tmp);
const trendDelta = (prior7.length && last7.length) ? last7Avg - prior7Avg : null;
const trendDirection = trendDelta === null ? "flat" :
  trendDelta > 0.15 ? "rising" :
  trendDelta < -0.15 ? "falling" :
  "steady";
const trendArrow = trendDirection === "rising" ? "?" :
  trendDirection === "falling" ? "?" :
  "?";
const trendColor = trendDirection === "rising" ? "#B03823" :
  trendDirection === "falling" ? "#537F1C" :
  "#705C57";

```

```

    ``
    ```js
 // Year-over-year
 const dataPrior = parsed
 .filter(d => d.date?.getFullYear() === priorYear && d.tmp !== null)
 .sort((a, b) => a.date - b.date);

 const doyOf = d => d3.utcDay.count(d3.utcYear(d), d);
 const latestDoy = latest ? doyOf(latest.date) : null;
 const sameDayPrior = latestDoy !== null
 ? dataPrior.find(d => doyOf(d.date) === latestDoy)
 : undefined;
 const yoyDelta = (sameDayPrior && latest) ? latest.tmp - sameDayPrior.tmp : null;

 const dataPriorToDate = latestDoy !== null
 ? dataPrior.filter(d => doyOf(d.date) <= latestDoy)
 : dataPrior;
 ``

    ```js
    // Full season table (scrollable)
    const recent = dataCurrent.slice().reverse();
    ``

    ```js
 // Status for current temp
 const status = latest?.tmp >= threshold ? "above" :
 latest?.tmp >= threshold - 1.8 ? "approaching" : // 1.8°F ? 1°C buffer
 "below";
 const statusColor = status === "above" ? "#B03823" :
 status === "approaching" ? "#F7941E" :
 "#537F1C";
 ``

    ```js
    const logo = await FileAttachment("assets/LOGO.png").url();

    display(htl.html`<header class="site-header">
      
      <nav class="site-nav">
        <a href="./">Overview</a>
        <a href="./season" aria-current="page">${currentYear} Season</a>
        <a href="./coolmix">Cool Mix Flows</a>
      </nav>
    </header>`);
    ``

    ```js
 display(htl.html`<div class="hero">
 <div class="hero-bg"></div>
 <p class="hero-eyebrow">Colorado River Mile ${RIVER_MILE} · Grand Canyon National
 Park</p>
 <h1 class="hero-title">${currentYear} Season Detail</h1>
 <p class="hero-sub">Current season trend and daily water temperature readings.</p>
 </div>`);
 ``

    ```js
    display(htl.html`<p class="last-updated">Data through ${latest?.date.toLocaleDateSt
    ring("en-US", {year: "numeric", month: "long", day: "numeric"})} · USGS, preliminar
    y</p>`);
    ``

    <div class="stat-row season-stats">
      <div class="stat-card stat-card--temp">

```

```

    <div class="stat-label">Current Temperature</div>
    <div class="stat-value" style=${{color: statusColor}}>${latest?.tmp.toFixed(1)}
°F</div>
    <div class="stat-sub" style=${{color: statusColor}}>${status === "above" ? "Above Threshold" : status === "approaching" ? "Approaching Threshold" : "Below Threshold"}</div>
  </div>
  <div class="stat-card stat-card--peak">
    <div class="stat-label">Peak This Season</div>
    <div class="stat-value">${seasonHigh?.toFixed(1)}°F</div>
    <div class="stat-sub">${seasonHighDate ? `Reached ${seasonHighDate.toLocaleDateString("en-US", {month: "short", day: "numeric"})}` : "?"}</div>
  </div>
  <div class="stat-card stat-card--days">
    <div class="stat-label">Days Above Threshold</div>
    <div class="stat-value">${daysAbove}</div>
    <div class="stat-sub">At or Above ${threshold.toFixed(1)}°F</div>
  </div>
  <div class="stat-card stat-card--trend">
    <div class="stat-label">Warming Trend</div>
    <div class="stat-value" style=${{color: trendColor}}>${trendArrow} ${trendDelta !== null ? Math.abs(trendDelta).toFixed(2) : "?"}°F</div>
    <div class="stat-sub">${trendDirection === "rising" ? "Warming vs. Prior Week" : trendDirection === "falling" ? "Cooling vs. Prior Week" : "Holding Steady"}</div>
  </div>
</div>

```js
// Tag each point with a run id so same-color points aren't bridged across gaps.
// A line segment connecting a point above the threshold to a point below it
// (or vice versa) is only ONE color in Plot ? coloring it by the endpoint's
// "above" flag makes part of that segment visually cross the threshold line
// in the wrong color. To fix this, we interpolate the exact date at which the
// series crosses the threshold and insert that crossing point as the shared boundary
// between the two runs. The segment before the crossing is then entirely on
// one side of the threshold, and the segment after is entirely on the other.
const dataCurrentRuns = (() => {
 const result = [];
 let runId = 0;
 dataCurrent.forEach((d, i) => {
 const isAbove = d.tmp >= threshold;
 if (i > 0) {
 const prev = dataCurrent[i - 1];
 const prevIsAbove = prev.tmp >= threshold;
 if (isAbove !== prevIsAbove) {
 // Linear interpolation for the date at which tmp === threshold
 const t = (threshold - prev.tmp) / (d.tmp - prev.tmp);
 const crossDate = new Date(prev.date.getTime() + t * (d.date.getTime() - prev.date.getTime()));
 // Close out the previous run exactly at the threshold
 result.push({ date: crossDate, tmp: threshold, runId, above: prevIsAbove });
 }
 runId++;
 // Start the new run at the same point, new color
 result.push({ date: crossDate, tmp: threshold, runId, above: isAbove });
 }
 result.push({ ...d, runId, above: isAbove });
 });
 return result;
})();

```js
// ?? Section 1: Current-year Temperature ??????????????????????????????
display(html`<div class="section-card">
  <div class="section-card-header">

```

```

    <h2 class="section-card-title">${currentYear} Daily Temperature Readings</h2>
  </div>
  <div id="chartCurrent-wrap"></div>
  <div class="inline-legend">
    <span class="il-item"><svg width="24" height="10"><line x1="0" y1="5" x2="24" y
2="5" stroke="#537F1C" stroke-width="2.5"/></svg> Below Threshold</span>
    <span class="il-item"><svg width="24" height="10"><line x1="0" y1="5" x2="24" y
2="5" stroke="#B03823" stroke-width="2.5"/></svg> Above Threshold</span>
    <span class="il-item"><svg width="24" height="10"><line x1="0" y1="5" x2="24" y
2="5" stroke="#B03823" stroke-width="1.5" stroke-dasharray="4,2"/></svg> ${threshol
d.toFixed(1)}°F Threshold</span>
  </div>
</div>`);

const chartCurrent = resize((width) => Plot.plot({
  width,
  height: 500,
  marginLeft: 50,
  marginBottom: 45,
  marginTop: 25,
  style: {
    fontFamily: "IBM Plex Mono, monospace",
    fontSize: "16px",
    background: "#ffffff",
  },
  x: {
    label: null,
    type: "time",
    domain: [d3.utcDay.offset(d3.min(dataCurrent, d => d.date), -2), d3.utcDay.offset
(d3.max(dataCurrent, d => d.date), 1)],
    ticks: d3.utcMonths(d3.utcMonth(d3.min(dataCurrent, d => d.date)), d3.utcDay.offse
t(d3.max(dataCurrent, d => d.date), 1)),
    tickFormat: d3.utcFormat("%b"),
  },
  y: { label: "Water temperature (°F)", domain: [toF(7), toF(22)] },
  marks: [
    Plot.rectY([{}], {
      x1: d3.min(dataCurrent, d => d.date),
      x2: d3.max(dataCurrent, d => d.date),
      y1: threshold, y2: toF(22),
      fill: "#B03823", fillOpacity: 0.07,
    }),
    Plot.ruleY([threshold], {
      stroke: "#B03823", strokeDasharray: "4 2", strokeWidth: 1.2,
    }),
    Plot.line(dataCurrentRuns, {
      x: "date", y: "tmp",
      z: "runId",
      stroke: d => d.above ? "#B03823" : "#537F1C",
      strokeWidth: 3,
    }),
    Plot.tip(dataCurrent, Plot.pointerX({
      x: "date", y: "tmp",
      title: d => `${d.date.toLocaleDateString()} \n ${d.tmp.toFixed(1)}°F`,
    })),
    Plot.dot(dataCurrent.slice(-1), {
      x: "date", y: "tmp",
      r: 5, fill: "#2C0E09", stroke: "#ffffff", strokeWidth: 1,
    }),
  ],
}));

document.getElementById("chartCurrent-wrap").replaceWith(chartCurrent);
```

```js
// ?? Trend summary between charts ?????????????????????????????????????
const trendSummary = yoyDelta !== null

```

```

    ? `Compared to this time last year, ${currentYear} is ${Math.abs(yoyDelta).toFixed(1)}°F ${yoyDelta > 0 ? "warmer" : "cooler"}.`
    : "Year-over-year comparison not available for this date.";

display(htl.html`<p class="trend-summary">${trendSummary}</p>`);
```

```js
// ?? Section 2: Season Comparison ?????????????????????????????????????????
display(htl.html`<div class="section-card section-card--secondary">
  <div class="section-card-header">
    <h2 class="section-card-title">Season Comparison</h2>
    <p class="section-card-sub">${currentYear} vs. ${priorYear}, same period</p>
  </div>
  <div id="yoy-chart-wrap"></div>
  <div class="inline-legend">
    <span class="il-item"><svg width="24" height="10"><line x1="0" y1="5" x2="24" y
2="5" stroke="#537F1C" stroke-width="2.5"/></svg> Below Threshold</span>
    <span class="il-item"><svg width="24" height="10"><line x1="0" y1="5" x2="24" y
2="5" stroke="#B03823" stroke-width="2.5"/></svg> Above Threshold</span>
    <span class="il-item"><svg width="24" height="10"><line x1="0" y1="5" x2="24" y
2="5" stroke="#8C7A76" stroke-width="2"/></svg> ${priorYear}</span>
    <span class="il-item"><svg width="24" height="10"><line x1="0" y1="5" x2="24" y
2="5" stroke="#B03823" stroke-width="1.5" stroke-dasharray="4,2"/></svg> ${threshol
d.toFixed(1)}°F Threshold</span>
  </div>
</div>`);

// Same run-splitting approach as the current-year chart above, so the
// ${currentYear} line here uses the same above/below-threshold coloring
// instead of a single flat color.
const dataPriorRuns = (() => {
  const result = [];
  let runId = 0;
  dataCurrent.forEach((d, i) => {
    const isAbove = d.tmp >= threshold;
    if (i > 0) {
      const prev = dataCurrent[i - 1];
      const prevIsAbove = prev.tmp >= threshold;
      if (isAbove !== prevIsAbove) {
        const t = (threshold - prev.tmp) / (d.tmp - prev.tmp);
        const crossDate = new Date(prev.date.getTime() + t * (d.date.getTime() - pr
ev.date.getTime()));
        result.push({ date: crossDate, tmp: threshold, runId, above: prevIsAbove });
      }
      runId++;
      result.push({ date: crossDate, tmp: threshold, runId, above: isAbove });
    }
    result.push({ ...d, runId, above: isAbove });
  });
  return result;
})();

const yoyChart = resize((width) => Plot.plot({
  width,
  height: 320,
  marginLeft: 50,
  marginBottom: 40,
  marginTop: 20,
  style: {
    fontFamily: "IBM Plex Mono, monospace",
    fontSize: "16px",
    background: "#ffffff",
  },
  x: {
    label: null,
    tickFormat: doy => {

```

```

        const monthStarts = [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334];
        const monthNames = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
        const i = monthStarts.indexOf(doy);
        return i >= 0 ? monthNames[i] : null;
    },
    ticks: [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334],
  },
  y: { label: "Water temperature (°F)", domain: [toF(7), toF(22)] },
  marks: [
    Plot.rectY([{}], {
      x1: 0, x2: 365, y1: threshold, y2: toF(22),
      fill: "#B03823", fillOpacity: 0.07,
    }),
    Plot.ruleY([threshold], {
      stroke: "#B03823", strokeDasharray: "4 2", strokeWidth: 1.2,
    }),
    Plot.line(dataPriorToDate, {
      x: d => doyOf(d.date), y: "tmp",
      stroke: "#8C7A76", strokeWidth: 2,
    }),
    Plot.line(dataPriorRuns, {
      x: d => doyOf(d.date), y: "tmp",
      z: "runId",
      stroke: d => d.above ? "#B03823" : "#537F1C",
      strokeWidth: 3,
    }),
    Plot.tip(dataCurrent, Plot.pointerX({
      x: d => doyOf(d.date), y: "tmp",
      title: d => `${currentYear} ? ${d.date.toLocaleDateString()} \n ${d.tmp.toFixed(1)} °F`,
    })),
    Plot.dot(dataCurrent.slice(-1), {
      x: d => doyOf(d.date), y: "tmp",
      r: 5, fill: "#2C0E09", stroke: "#ffffff", strokeWidth: 1,
    }),
  ],
});

```

```

document.getElementById("yoy-chart-wrap")?.replaceWith(yoyChart);
```

```

```

```js
// ?? Section 3: Recent Observations ?????????????????????????????????
display(htl.html`<div class="section-card section-card--table">
  <div class="section-card-header">
    <h2 class="section-card-title">Recent Observations</h2>
    <p class="section-card-sub">${dataCurrent.length} readings this season</p>
  </div>
  <div class="recent-table-scroll">
    <table class="recent-table">
      <thead>
        <tr>
          <th>Date</th>
          <th>Temp (°F)</th>
          <th>vs. prev</th>
          <th>Flow (cfs)</th>
        </tr>
      </thead>
      <tbody>
        ${recent.map((d, i) => {
          const prev = recent[i + 1];
          const delta = prev ? d.tmp - prev.tmp : null;
          const arrow = delta === null ? "" :
            delta > 0.1 ? "?" :
            delta < -0.1 ? "?" :
            "?";
          const arrowColor = delta === null ? "#8C7A76" :
            delta > 0.1 ? "#B03823" :

```

```

        delta < -0.1 ? "#537F1C" :
            "#8C7A76";
    return htl.html`<tr class=${d.tmp >= threshold ? "row-above" : ""}>
        <td>${d.date.toLocaleDateString("en-US", {month: "short", day: "numeric"})
    }</td>
        <td class="cell-temp">${d.tmp.toFixed(1)}</td>
        <td class="cell-delta" style=${color: arrowColor}>${arrow} ${delta !==
null ? Math.abs(delta).toFixed(1) : "?"}</td>
        <td class="cell-flow">${d.cfs ? d.cfs.toLocaleString() : "?"}</td>
    </tr>`;
    }}}
</tbody>
</table>
</div>
</div>`;
```

```js
display(htl.html`
    <footer class="site-footer">
        <p class="site-footer-citation">
            Observations from
            <a href="https://waterdata.usgs.gov/monitoring-location/USGS-09380000"
                target="_blank"
                rel="noopener noreferrer">
                USGS Gage 09380000
            </a>
            • Temperature estimates based on
            <a href="https://doi.org/10.1002/eap.2279"
                target="_blank"
                rel="noopener noreferrer">
                Dibble et al. (2020)
            </a>
            thermal model.
        </p>
        <p class="site-footer-credit">
            Graphs may be used for non-commercial purposes provided that they are not alt
            ered or edited and are appropriately credited to the Grand Canyon Trust.
        </p>
    </footer>
`);
```

```